FOOD STORE · PROYECTO INTEGRADOR

Trabajo Práctico N.º 8 (TP10) · Unidad 8: Bases de Datos para Dispositivos Móviles e Integración

Asignatura: Base de Datos II (UTN TUP)
Estudiante: Cristian Rodriguez (Cloverfield)
Fecha: Noviembre 2026
Rama Git: `tp10-moviles-integracion`
Motores: PostgreSQL 16+ · SQLite 3.50+
NoSQL: MongoDB 9.0+ (Unidad 7)
Herramientas: Kiro (EARS) · OpenCode
Entrega: Semana 13 / Cierre de Cursada

1. La Base de la Tablet

1.a Qué datos viajan al dispositivo móvil

Para operar el food truck en una feria gastronómica sin señal de red garantizada, la base de datos embebida SQLite en la tablet debe contener la información mínima necesaria para atender clientes y armar borradores. Se aplica la **Regla de Oro:** *"El móvil propone, el servidor dispone"*.

Entidad / Atributo	¿Viaja?	Justificación Técnica, Operativa y Legal
`producto (id, nombre)`	SÍ	Imprescindible para confeccionar la carta y exhibirla al público en la interfaz táctil.
`producto (precio)`	SÍ	Se almacena como `precio_centavos INTEGER` para evitar imprecisiones de coma flotante.
`producto (stock)`	SÍ	Viaja como `stock_visto INTEGER` con carácter puramente orientativo; no reserva unidades.
`producto (disponible)`	SÍ	Permite ocultar del menú productos fuera de servicio (`disponible_visto IN (0,1)`).
`producto / cat (eliminado)`	SÍ	Viaja como **lápida (*tombstone*)**: oculta el producto del catálogo sin quebrar integridad referencial.
`usuario (contraseñas/hashes)`	NO	Crítico. Nunca abandonan PostgreSQL para evitar filtraciones masivas ante robo o pérdida de la tablet.
`cliente (email, celular, datos)`	NO	Datos personales protegidos por la Ley 25.326 y principio de minimización; innecesarios en mostrador.
`pedido (historial global)`	NO	La tablet no realiza auditorías ni balances históricos; minimiza el consumo de batería y almacenamiento.

Fundamentación legal y fuente de verdad: Se aplica el principio de mínimo privilegio y minimización de datos (Art. 4 y 9 de la Ley 25.326). Lo que no se almacena en el dispositivo móvil no puede ser sustraído. **PostgreSQL** es la única fuente de verdad (*Single Source of Truth*); la tablet solo prepara borradores temporales y nunca altera stock ni precios centrales de manera autónoma.

1.b Esquema SQLite en modo STRICT y Restricciones

El esquema en `movil/schema_sqlite.sql` se ejecuta con `PRAGMA foreign_keys = ON;` y estructura cuatro tablas bajo estándar `STRICT`:

- `producto_cache`: Catálogo de lectura con precios enteros en centavos y banderas enteras (0/1).
- `pedido_borrador`: Identificado mediante `id_local TEXT PRIMARY KEY` (UUID generado en la tablet).
- `item_borrador`: Clave foránea con `ON DELETE CASCADE` al borrador, clave foránea a producto y `CHECK (cantidad > 0)`.
- `cola_sync`: Identificador `solicitud_uuid TEXT PRIMARY KEY` con estados `('PENDIENTE', 'CONFLICTO', 'RECHAZADO', 'SIMULADO_APROBADO')` y el índice condicional único `ux_cola_una_pendiente` sobre `id_local WHERE estado = 'PENDIENTE'`.
- `v_catalogo_disponible`: Vista que filtra `eliminado = 0 AND disponible_visto = 1` y divide por 100.0.

1.c Validación Transaccional y Casos C1 y C7

Se ejecutó la suite de pruebas en `movil/pruebas_sqlite.sql` comprobando los tres escenarios requeridos:

- **Transacción atómica:** Un borrador completo (cabecera + ítems + solicitud en cola) se asienta en un solo bloque `BEGIN ... COMMIT`. Una segunda operación se cancela con `ROLLBACK`, verificándose 0 filas persistidas.
- **Caso C1 (Catálogo Offline):** La vista `v_catalogo_disponible` devuelve los productos aptos (Pizza Muzzarella, Empanadas, Gaseosa), excluyendo la Fugazzeta (eliminada) y Cerveza (no disponible).
- **Caso C7 (Datos Inválidos):** Un intento de insertar `cantidad = 0` es rechazado por `CHECK constraint failed: cantidad > 0`, y un intento con producto inexistente (ID 9999) es rechazado por `FOREIGN KEY constraint failed`. Registros huérfanos: **0**.

2. Sincronizar y Resolver: El Semáforo

2.a Instantánea del Servidor e Importación

El servidor central PostgreSQL genera la instantánea oficial mediante la vista `v_snapshot_movil` (script `movil/postgresql_snapshot.sql`). Los precios se convierten a centavos enteros en origen (`(p.precio * 100)::INTEGER`) y la baja lógica calcula la lápida: `(NOT p.activo OR NOT c.activa) AS eliminado`. Como `disponible` y `eliminado` son complementos lógicos estrictos, nunca son simultáneamente verdaderos ni falsos. La vista se exporta directamente a `movil/servidor_snapshot.csv` con marca de tiempo precisa en microsegundos y se carga en `snapshot_servidor` en SQLite, recargando `producto_cache` mediante la consulta de sincronización oficial.

2.b Lógica del Semáforo de Sincronización

El script `movil/semaforo.sql` implementa una Common Table Expression (CTE) que compara solicitudes en estado `PENDIENTE` contra `snapshot_servidor`:

- ROJO (RECHAZADO): Producto inexistente, lápida activa (`eliminado = 't'`), no disponible para venta o cantidad solicitada superior al stock.
- AMARILLO (CONFLICTO): No existe causal de rechazo, pero el precio visto difiere del precio vigente en el servidor.
- VERDE (SIMULADO_APROBADO): Todos los ítems coinciden con el inventario y lista de precios del servidor.

Regla de Precedencia e Idempotencia: El ROJO invalida cualquier CONFLICTO. Solo se procesan filas `PENDIENTE`; reejecutar la simulación no modifica lo ya resuelto.

2.c Evidencia y Comprobación de Casos de Sincronización (C2 a C6)

Caso	Situación de Prueba	Resultado del Semáforo	Resolución y Comportamiento Verificado
Caso C5	Borrador idéntico al inventario del servidor (2 Empanadas a $1.200).	SIMULADO_APROBADO	Todos los ítems coinciden en stock, disponibilidad y tarifa. Indica factibilidad sin constituir una venta real.
Caso C2	Borrador con Muzzarella a $9.000. En PostgreSQL se actualiza el precio: `UPDATE producto SET precio = 105.00` (`servidor_casos_c2_c4.sql`) y se reexporta la instantánea.	CONFLICTO ↓ SIMULADO_APROBADO	**Envío 1:** CONFLICTO tarifario. **Resolución:** Cliente acepta $10.500. La tablet **recarga el catálogo oficial** desde `snapshot_servidor`, actualiza el ítem a 1050000 centavos y un segundo envío resulta aprobado.
Caso C3	Borrador por 20 Pizzas Napolitanas; stock del servidor en 8 unidades.	RECHAZADO	Rechazo inmediato por quiebre de stock. La tablet no emite la orden y el stock de PostgreSQL permanece inalterado.
Caso C4	Borrador con Empanada Jamón y Queso. En PostgreSQL se aplica baja lógica: `UPDATE producto SET activo = FALSE` (`servidor_casos_c2_c4.sql`) exportando lápida (`disponible='f', eliminado='t'`).	RECHAZADO	**Semáforo:** RECHAZADO por lápida activa. **Recarga Oficial:** La tablet **no marca a mano** en SQLite; ejecuta la recarga oficial (`INSERT OR REPLACE INTO producto_cache SELECT ... FROM snapshot_servidor`), ocultándolo de `v_catalogo_disponible` pero preservándolo en caché.
Caso C6	Reenvío de solicitud C5 con el mismo UUID por corte de señal.	REINTENTO RECHAZADO (PK)	SQLite rechaza la inserción por `UNIQUE constraint failed: cola_sync.solicitud_uuid`. El semáforo no reevalúa la orden previa (idempotencia).

Dónde va la protección real en producción: La protección definitiva contra duplicaciones no puede recaer en el dispositivo móvil (que puede ser reseteado o manipulado). En un entorno productivo, reside en **PostgreSQL** mediante una tabla central `solicitud_movil (solicitud_uuid UUID PRIMARY KEY, ...)` insertada dentro de la misma transacción de `sp_crear_pedido` con `ON CONFLICT (solicitud_uuid) DO NOTHING`.

Comprobación de Invariabilidad en PostgreSQL (Consigna 2.c): Se documentó y comprobó en `movil/servidor_casos_c2_c4.sql` mediante `SELECT COUNT(*) FROM pedido` y `SELECT SUM(stock) FROM producto` que el inventario consolidado y los pedidos confirmados en PostgreSQL permanecieron idénticos antes y después de todas las pruebas en la tablet (*el móvil propone, el servidor dispone*).

3. Integrar y Revisar: La Capa de Datos Completa

3.a Mapa de la Capa de Datos (Tres Motores, Una Fuente de Verdad)

Food Store articula tres motores complementarios con fronteras de responsabilidad estrictas:

Motor de Base de Datos	Qué información guarda	Quién manda sobre cada dato	Unidad
PostgreSQL 16+ (Oficina Central)	Catálogo maestro, inventario transaccional, pedidos confirmados, detalles de venta, usuarios con contraseñas hasheadas (`pgcrypto`) y auditoría.	**Fuente de verdad única (*Single Source of Truth*)**. Posee autoridad absoluta sobre precios oficiales, stock y confirmación de ventas (ACID con `FOR UPDATE`).	U1 a U6
MongoDB 9+ (Capa Documental)	Colección desnormalizada `pedidos` (*Aggregate Root*), conteniendo cabecera, snapshot del cliente, array de ítems y subtotales precalculados.	**Autoridad delegada de lectura agregada**. No descuenta stock ni fija precios; provee consultas analíticas de alta velocidad sin sobrecargar el OLTP.	U7
SQLite 3.50+ (Tablet Food Truck)	Copia local del catálogo (`producto_cache`), borradores de órdenes (`pedido_borrador` , `item_borrador`) y cola de envíos (`cola_sync`).	**Sin autoridad decisoria**. Libreta desconectada para preparar borradores temporales. Jamás modifica stock central: *el móvil propone, el servidor dispone*.	U8

3.b Plan de Mantenimiento Integral

- **Respaldo y prueba de restauración:** En PostgreSQL se generan volcados nativos diarios comprimidos (`pg_dump -Fc`) en `db/backups/` . Se prueban mensualmente mediante el script automatizado `restore_test.ps1` en la base estéril `foodstore_restore_test` , verificando integridad y midiendo RTO y RPO. Las tablets no requieren backups fríos centralizados: son reemplazables y descargan una instantánea fresca del servidor.
- **Restauración de backup viejo:** Las tablets quedarían con datos "del futuro". El protocolo exige purgar inmediatamente la tabla `snapshot_servidor` en las terminales móviles, descargar la vista `v_snapshot_movil` restaurada y reejecutar el semáforo sobre todas las solicitudes en estado `PENDIENTE` antes de procesarlas.
- **Revisión de solicitudes en conflicto o rechazo:** Revisión diaria al cierre de turno por el **Encargado del Food Truck**. Las solicitudes en CONFLICTO se resuelven en el momento con el comensal; las solicitudes en RECHAZADO se auditan al final del día para ajustar el stock físico y notificar a compras.

3.c Code Review de la Cursada con Inteligencia Artificial

Se auditaron los scripts del proyecto con el modelo de IA en modo plan/revisión, analizando tres hallazgos de alto impacto:

ID / U	Componente Auditado	Propuesta de la IA y Comprobación	Decisión Técnica y Fundamento
H1 (U8)	`movil/schema_sqlite.sql` (Tabla `cola_sync`)	**Propuesta:** Crear `UNIQUE (id_local)` . **Comprobación:** Al probar el Caso C2, la tablet debió emitir un segundo envío tras aceptar el cliente. La inserción falló con `UNIQUE constraint failed` .	DESCARTADO CON FUNDAMENTO Un borrador puede requerir reintentos. Se adoptó el índice parcial `CREATE UNIQUE INDEX ux_cola_una_pendiente ON cola_sync (id_local) WHERE estado = 'PENDIENTE'` , impidiendo colas paralelas pero permitiendo reenvíos.
H2 (U1/U3)	`db/tp7_clase1_prerrequisitos.sql` (Proc. `sp_crear_pedido`)	**Propuesta:** Usar `FOR UPDATE` en validación de stock. **Comprobación:** Con sesiones concurrentes sobre `stock=1` , sin bloqueo ambas leyeron 1 e intentaron descontar, quebrando el CHECK de stock.	ACEPTADO Y VERIFICADO Se incorporó `SELECT stock INTO v_stock ... FOR UPDATE;` dentro del bloque transaccional, garantizando serializabilidad en compras concurrentes.
H3 (U5)	`db/tp7_clase2_seguridad.sql` (Roles de Soporte)	**Propuesta:** `GRANT SELECT ON ALL TABLES` para rol soporte. **Comprobación:** La tabla `usuario` guarda hashes bcrypt y teléfonos. Un GRANT ALL expone credenciales a operadores de soporte.	DESCARTADO CON FUNDAMENTO Viola mínimo privilegio y la Ley 25.326. El acceso de soporte se restringe a vistas de proyección como `v_usuarios_publico` (sin contraseñas).